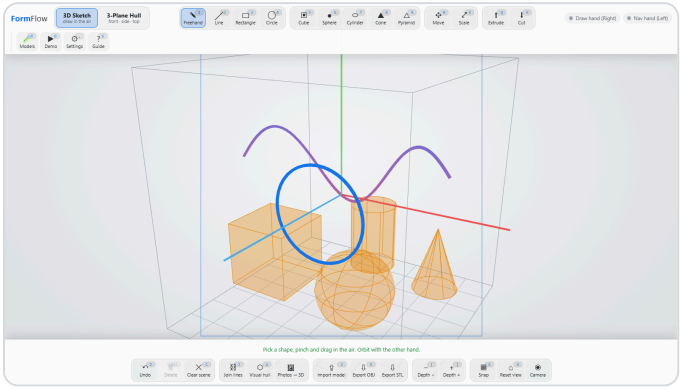
FormFlow

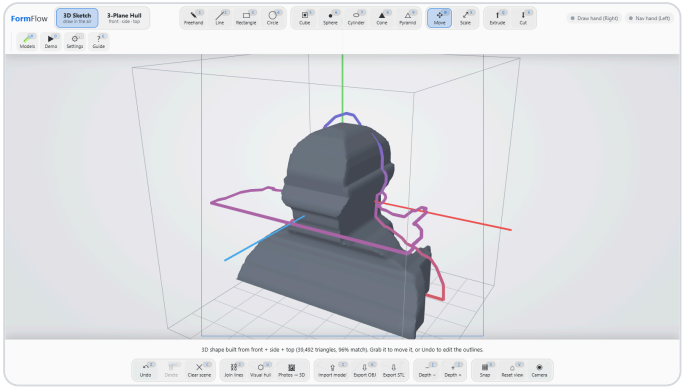
Sketch in the air. Build it in 3D.

FormFlow is a web app that lets a beginner make 3D shapes with their hands and an ordinary webcam. You pinch your fingers in the air to draw, or you give it three pictures of an object (front, side, top) and it builds the 3D shape for you.

Made by Saran S Kumar (453), solo project.



Drawing in 3D with hand gestures



Three pencil drawings of a head turned into a 3D bust

THE WHOLE PROJECT IN FIVE LINES

1. A webcam picture goes into an AI (made by Google) that finds **21 points on each hand**.
2. Our code turns those points into **gestures** (pinch = draw, fist = turn the view) and smooths out the shaking.
3. A small 3D engine draws what you make, so you can move, resize and export it.
4. For pictures, our code finds the **outline** of the object in each photo.
5. Three outlines are **crossed** (a "visual hull") to make a solid 3D shape you can save as OBJ or STL.

HOW TO USE THIS GUIDE

Each page answers one question. You do not need to know anything before you start. Words in **bold** are terms you may be asked. The last page is a cheat sheet with a glossary and likely viva questions.

CONTENTS

1 Name · 2 Problem and solution · 3 How it is built · 4 Stack · 5 to 7 How computer vision is used · 8 Conclusion and future scope · 9 Cheat sheet

THE PROBLEM AND OUR ANSWER

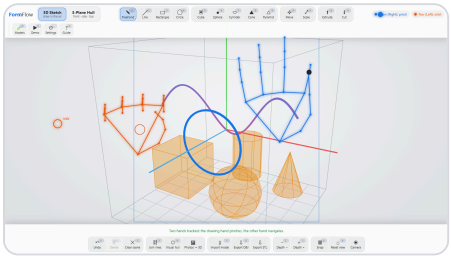
Problem: learning 3D is hard for beginners

- **CAD software needs training.** Menus, mouse tricks and lots of buttons make beginners give up before they see their idea in 3D.
- **Special hardware is a barrier.** Depth cameras, gloves or markers cost money and are not in a classroom.
- **The camera in every laptop is a normal 2D camera.** It sees flat pictures, so we need computer vision to get 3D out of them.

OUR GOAL

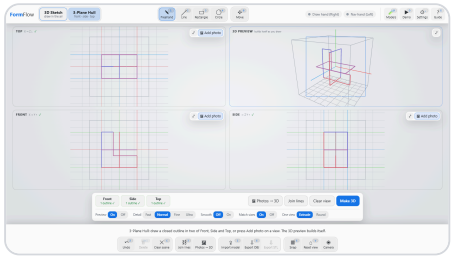
Make 3D easy and fun for beginners to learn. Rules we set ourselves: one ordinary webcam, no markers or gloves, no training of our own AI, real time, and everything runs in a web browser.

Solution: three easy ways to make a 3D shape



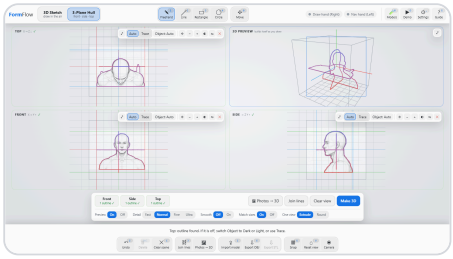
1 Sketch in the air

Two hands are tracked. The drawing hand pinches to draw; the other hand turns, slides and zooms the view.



2 Draw three views

Outline an object from the front, side and top. A live 3D preview builds itself.



3 Start from photos

Drop pictures in. The app finds the outline of the object and builds the 3D shape.

Gestures to remember

Gesture	How to make it	What it does
Pinch	touch thumb to index finger	draw, place a shape, press a button
Grab	touch thumb to middle finger	move an object
Fist (other hand)	close all fingers	turn (orbit) the view
Peace sign (other hand)	index and middle finger up	slide (pan) the view
Pinch with both hands	move the hands apart or together	zoom

MADE FOR BEGINNERS

A guided tour with a practice step that checks the camera works, a message after every action telling you what to do next, a **Demo** button with ready-made examples (rocket, house, vase, head), voice commands, undo, and a mouse mode if you have no camera.

WHAT YOU GET AT THE END

A 3D model you can move, resize, cut and extrude, and **export as OBJ or STL**, the standard files for 3D viewers and 3D printers. You can also import OBJ / STL models.

HOW IT IS BUILT

The big picture: two lanes, one 3D engine

Purple boxes are AI models made by Google. Blue boxes are code we wrote.

Lane A: from your hands to a pointer

Webcam live video, 30 pictures a second → **Hand + arm AI** 21 points per hand → **Tracker** which hand is which → **Gestures** pinch, grab, fist, peace → **Smoothing** removes shake → **3D engine** draws and edits

Lane B: from outlines to a 3D shape

Outlines drawn, or found in photos → **Tracer** background, object, edge → **Visual hull** cross the 3 views → **Surface nets** smooth mesh → **Export** OBJ and STL

The layers (from camera to screen)

Layer	What it does	Main files
Sensing	Opens the camera, runs the Google hand and arm models, can zoom in on small or far hands.	main.js, roi.js, forearm.js
Understanding	Keeps track of two hands, decides the gesture, smooths the pointer, maps it to the screen.	tracker.js, gestures.js, track.js, oneEuro.js, mapping.js
Engine	The 3D scene: camera, tools, undo, three-view workspace, outline finder, visual hull, meshing, extrude and cut.	sketch.js, planes.js, trace.js, hull.js, extrude.js, csg.js
Display	Draws the scene with WebGL, plus buttons, the guide and hints.	scene.js, hud.js, guide.js, planebar.js
Server	A small Python program that serves the page and saves exported files.	server/app.py

Life of one camera frame

1. The camera gives a picture. If hands are small or far, we crop and zoom on them first.
2. The AI returns 21 points per hand (and the wrist and elbow, every third frame).
3. The tracker removes duplicates and false hands and keeps each hand's identity.
4. The gesture code measures finger distances and decides pinch, grab, fist or peace.
5. The pointer is smoothed and mapped onto the screen. The controller turns it into an action.
6. The 3D engine changes the scene and Three.js draws it, all in about 1/30 of a second.

DESIGN RULES THAT HELPED

The thinking code never touches the screen or the camera, so we test it without either (**288 automated tests**). Time is passed in as a number, so tests are exact. If a hand is lost mid-stroke the stroke is cancelled, never invented.

OWN CODE VS BORROWED

Borrowed: two Google AI models, Three.js, FastAPI. **Our own:** everything else, about **6,700 lines** of JavaScript (4,100 of them in the thinking layers) and 190 of Python. No AI was trained by us.

STACK

What was used, where it comes from, and where computer vision comes in

Part	Where from	What it does here	Computer vision link
MediaPipe Tasks Vision (Hand Landmarker + Pose Landmarker)	Google, already trained, runs in the browser (WebAssembly, GPU if available)	Finds 21 points on each hand, in the picture and in 3D, and the wrist and elbow of the arm.	AI convolutional neural networks (CNNs); detection, then landmark regression
JavaScript modules (our code)	Written for this project	Tracker, gestures, filters, sketch engine, outline finder, visual hull, meshing.	Classical CV filtering, thresholding, segmentation, geometry
Three.js (WebGL)	Open source	Draws the 3D scene, and four views side by side in the three-view mode.	Geometry the same pinhole camera model we use for unprojection
Camera + Canvas	Built into Chrome and Edge	Gives the live video, and lets us read the pixels of a photo.	Images are grids of numbers (pixels)
FastAPI + Uvicorn (Python 3.12)	Open source	Runs the app on your own computer, saves exports and settings, serves the sample pictures.	None: no vision in the server
Web Speech	Built into the browser	Voice commands such as "undo" or "cube".	None (speech, not vision)
Node test, pytest	Standard tools	265 JavaScript tests + 23 Python tests.	Tests use simulated hands and shaded pictures

Why these choices

- **Browser, not a desktop app:** nothing to install for the user, runs on any laptop, and the first Python version was too laggy.
- **Pretrained AI:** training a hand model needs huge data. A student project should use the best sensor available and spend the effort on what comes after it.
- **Classical CV for the rest:** every step can be explained, tested and drawn on a slide. No black box between the outline and the 3D shape.
- **Offline:** the models and libraries live in the project's `vendor` folder, so it works without internet.

How to run it

1. Double-click `run.bat` (first run installs the Python parts).
2. Open `http://127.0.0.1:8000` in Chrome or Edge.
3. Allow the camera. Follow the guide, pass the practice step.
4. Run `run.bat test` to run all 288 tests.

Add `?demo=photohull&set=head` to the address to open a ready-made scene.

WHAT IS "PRETRAINED"?

A neural network learns from millions of examples (this is **training**). Afterwards it only needs one picture and gives an answer in a few milliseconds (this is **inference**). Google did the training. We only do inference, so we do not need any training data or a powerful computer.

HOW COMPUTER VISION IS USED · PART 1 OF 3

Seeing the hands and understanding gestures

First: what is a picture?

A picture is a grid of **pixels**. Each pixel is a number: 0 is black, 255 is white. A colour pixel has three numbers (red, green, blue). A video is about 30 pictures per second, called **frames**. Computer vision is the art of getting meaning out of these numbers.

	0	1	2	3	4	5
0	255	255	200	40	40	255
1	255	220	60	30	50	255
2	255	120	20	20	80	255
3	255	210	70	40	160	255
4	255	255	230	200	255	255
5	255	255	255	255	255	255

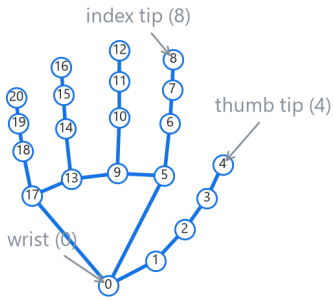
A tiny 6 by 6 picture and its numbers

Finding the hand with AI

A **convolutional neural network (CNN)** is a stack of small filters that slide over a picture and learn to spot patterns: edges, then fingers, then a whole hand. Google's hand model works in two steps:

1. **Palm detector:** one quick look at the whole picture finds where hands are (a "single-shot" detector, the same idea as SSD and YOLO).
2. **Landmark network:** looks closely at each hand and gives **21 points**: x, y in the picture and a 3D position in metres.

It also says if the hand is left or right and how sure it is. A second, lighter model finds the **wrist and elbow** and helps keep a hand when the fingers are hard to see. For far-away hands we **crop and zoom** on them before detecting.



The 21 landmarks

Turning points into gestures

Distances in the picture change when the hand is near or far. So we divide every distance by the **palm size** (wrist to the knuckle of the middle finger, points 0 and 9):

```
pinch ratio = distance(thumb tip 4, index tip 8) ÷ palm size pinch when ratio < 0.30, release when > 0.45
```

- **Grab** uses thumb tip and middle fingertip. **Fist**: all four fingertips are closer to the wrist than their middle joint. **Peace**: index and middle up, ring and pinky curled.
- **Hysteresis:** two different thresholds (0.30 to start, 0.45 to stop) stop the gesture flickering on and off at the edge.
- **Debounce:** a gesture must last 2 to 3 frames, so one bad frame does nothing.
- **3D landmarks:** turning your hand sideways makes a flat picture think your fingers moved closer. The 3D (world) points do not have this problem.
- **Personal thresholds:** in practice mode the app measures your open and pinched hand (two groups of numbers) and sets the limits between them.
- **Two roles:** the drawing hand (you choose which) draws and presses; the other hand only navigates. This avoids accidents.

SYLLABUS TOPICS TOUCHED

Pretrained models and convolutional networks, classification with a few classes (rule-based), convolution ideas. We use AI for sensing but do **not** fine-tune it.

EXAM ANSWER IN ONE LINE

"We use Google's pretrained CNN to get 21 hand landmarks, then classify gestures with palm-normalised distance ratios, hysteresis and debounce."

Cleaning the signal and finding objects in pictures

Noise and filters

Real signals are shaky: a hand trembles and the camera adds **noise**. A **low-pass filter** averages the recent past and so removes fast wiggles, but strong filtering makes the pointer lag. The **One-Euro filter** fixes this: it smooths a lot when you move slowly and very little when you move fast. It speeds up when the measured speed (a **finite difference**: change in position ÷ time) is high.

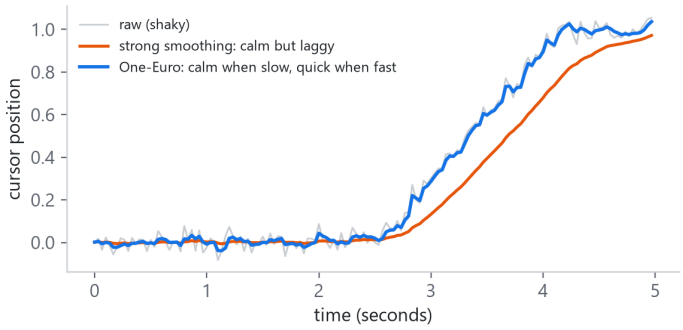
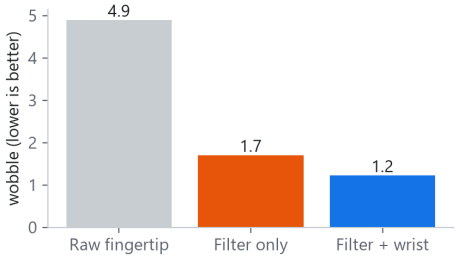


Illustration: the same shaky signal, filtered two ways

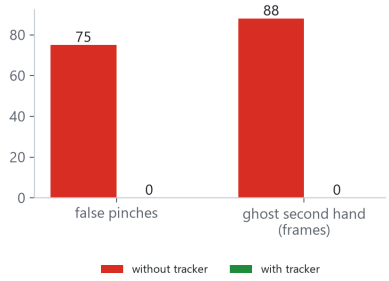


Measured on a simulated shaky hand

- **Wrist anchor:** fingertips shake about 5 times more than the wrist, so the cursor follows the steady palm and only the small pinch offset is filtered. Result: wobble fell from **4.9 to 1.2**.
- **Glitch gate:** a jump of more than 18% of the picture in one frame is a tracking error, not a movement, and is ignored.
- **Pinch latch:** closing your fingers drags the cursor a little, so on a pinch start we go back 0.15 s and hold the earlier position. Strokes start where you aimed.

Making the AI dependable: the tracker

What goes wrong	What we do
Left and right labels flip	follow the hand by position
One hand is seen twice	merge the copies
A false hand for one frame	wait 3 frames to be sure
Hand lost for a moment	hold the last pose 0.25 s (0.5 s if the arm is seen)



300 simulated frames with glitches added

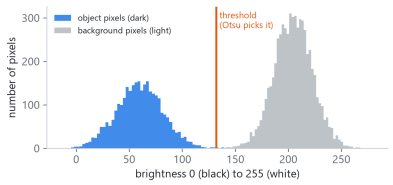
Segmentation: finding the object in a photo

Segmentation means labelling which pixels belong to the object. We do it with classical methods, in five stages:



1 Picture **2** Guess the background **3** What differs **4** Biggest blob, filled **5** Edge, 41 points

- **Background model:** blend the colours along the four picture borders to guess the empty background (this follows uneven light). Pixels that differ from it are "object" (this is **background subtraction**).
- **Otsu threshold:** picks the brightness cut that best splits two groups of pixels. It is the same as **k-means clustering** with two clusters on one number.
- **Connected components** keep the biggest blob; **hole filling** makes it solid (a Gestalt idea: closure).
- **Boundary walk + Douglas-Peucker** trace the edge and keep only the corner points.



Two groups of pixels, one threshold

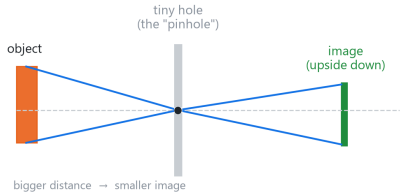
Quality is measured with IoU = overlap ÷ union of our outline and the true shape (1.0 is perfect). On simulated shapes with uneven light and noise we score about **0.98**.

From a flat camera to a 3D shape

The pinhole camera

Light from a point passes through one tiny hole and lands on the image, upside down. Far things look small. Our 3D engine uses the same maths to draw the scene and to go back from a screen point to a ray in 3D (**unprojection**).

```
screen_x = c_x + f · (x ÷ z)   f = 1.35 × window height
field of view = 2·atan(1 ÷ 2.7) = 40.6°
```

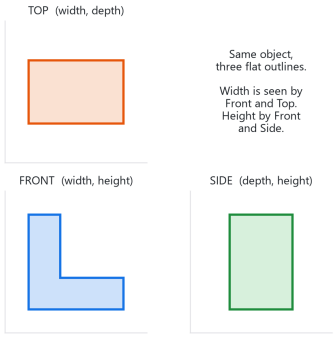


Pinhole camera model

Drawing on glass: the ray from your cursor is cut by an invisible flat plane facing the camera. That gives the 3D point. You move the plane nearer or farther with **Depth**.

Why one camera cannot see depth, and what we do instead

A normal camera has no **stereopsis** (the depth you get from two eyes). One picture cannot tell a small near object from a big far one. A hand does shrink with distance, so palm size is a rough depth cue, but not a measurement. For object shape we use the **three-view idea of engineering drawings**:



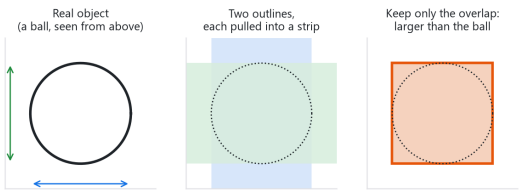
One object, three outlines

- Front shows width and height, Side shows depth and height, Top shows width and depth.
- Every size is seen by two views, so they **check each other**. If they differ by over 15% we warn you, and we stretch the views to agree.
- Each outline drawn on its own flat plane is a true silhouette, with no guessing.

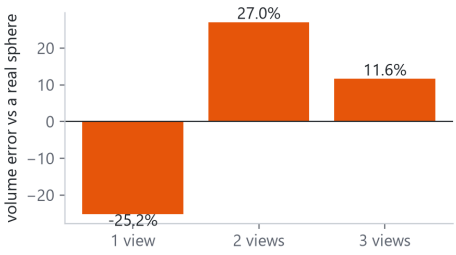
Visual hull: crossing the outlines

Pull each outline straight through space like cookie-cutter dough. The real object must fit inside **all** of these tubes, so we keep only where they **overlap**.

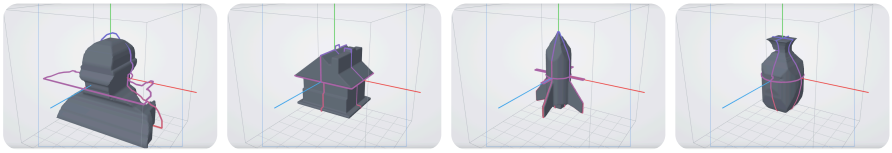
1. Align the views' shared sizes.
2. Fill each outline into a mask (even-odd rule).
3. Make a 3D grid of small cubes (**voxels**). A voxel is solid only if **every view** says solid.
4. Keep the biggest connected piece; blur it a little.
5. **Surface nets** turn the solid into a smooth triangle mesh. Export as OBJ or STL.



A hull is never smaller than the object, so it can be too big



Case study: a ball. Two views over-estimate by 27%, three by 12%. The maths predicts 27.3% and 11.9%, so our code is correct.



Head, 96% match | House | Rocket | Vase (1 photo)

Speed: a 64-cube grid builds in about 50 ms; a fine 96-cube grid in about 160 ms. A hull can fill in dents but never carve them, which is why a face looks blocky.

CONCLUSION AND FUTURE SCOPE

Conclusion

FormFlow shows that a **normal webcam, one pretrained AI and classical computer vision** are enough to make 3D modelling easy and fun for a beginner. Hands become the mouse, simple outlines or photos become a 3D model, and everything runs in a browser with no special hardware.

LESSON 1

Cleaning the AI's output matters as much as the AI. Filters and a tracker turned flickering landmarks into a usable pointer (false pinches 75 → 0, wobble 4.9 → 1.2).

LESSON 2

Geometry gives 3D without a depth sensor. Three outlines are enough, and our error matches the theory.

LESSON 3

Make it testable. Keeping thinking code separate from camera and screen gave 288 tests that run in seconds.

Limits (be honest in the viva)

- A **visual hull** cannot carve dents or hollows and always over-estimates. A face from three drawings looks like a blocky bust.
- **One camera gives no real depth:** sizes are relative, not in millimetres.
- Gesture limits were tuned by hand with simulated and practice data. **Real-camera tuning is still ongoing**; our numbers on tracking and filtering come from simulated hands.
- One object per picture, on a fairly plain background. Faint pencil on a busy background can fail.
- Not covered from the syllabus: **stereo with two cameras, Sobel or Canny edge detection, template matching, fine-tuning, YOLO or U-Net models**. We used related ideas instead (three views, mask edges, ratios, pretrained sensors).

Future scope

Idea	What it would add
More human-looking models	Project the photos onto the mesh as colour; add a depth estimate from the picture for the nose and brow; use a face-landmark model to fit a head template. This is the best next step for faces.
Two webcams (stereo)	Real depth in millimetres from **binocular disparity** and the epipolar constraint.
Sobel / Canny edge mode	Outlines for low-contrast photos, where thresholding fails.
Learned gesture classifier	Record real sessions (a session log already exists), train a small model, and compare with our rules.
Fine-tuned or AI image-to-3D	An optional AI mode (for example TripoSR or Hunyuan3D behind the Python server) for lifelike results from one photo.
Scenes and colour	Several objects per picture, colours and textures, save and reopen projects, and a classroom mode with lesson steps for beginners.

IF YOU REMEMBER ONE THING

"FormFlow lets a beginner build 3D shapes with their hands or three pictures. A pretrained AI finds the hands, our computer-vision code cleans and understands the signal and finds object outlines, and a visual hull crosses three outlines into a 3D model."

CHEAT SHEET

Glossary and likely questions

Glossary

Pixel	One number (or three for colour) in a picture's grid.
Frame	One picture of a video (about 30 a second).
Landmark	A named point, such as the index fingertip.
CNN	Neural network built from sliding filters, good at pictures.
Pretrained	Already trained by someone else; we only run it.
Inference	Using a trained network to get an answer.
Detector	Finds where objects are (a box) in one pass.
Noise	Random small errors in a signal.
Low-pass filter	Removes fast wiggles, keeps slow change.
One-Euro	Low-pass filter whose strength depends on speed.
Hysteresis	Different limits to switch on and off, to stop flicker.
Debounce	Ignore a change unless it lasts a few frames.
Threshold	A cut-off number that splits things in two.
Otsu	Method that finds the best threshold from a histogram.
Segmentation	Deciding which pixels belong to an object.
Silhouette	The filled outline shape of an object.
IoU	Overlap ÷ union of two shapes (1 = perfect).
Pinhole camera	Simplest camera model: points map through one hole.
Unprojection	Going from a screen point back to a 3D ray.
Stereopsis	Depth from two viewpoints (two eyes or cameras).
Orthographic view	Flat view with no perspective: front, side, top.
Voxel	A small 3D cube, the 3D version of a pixel.
Visual hull	Shape kept where all views' silhouettes overlap.
Mesh	A surface made of triangles. Saved as OBJ or STL.
Surface nets	Turns a voxel solid into a smooth mesh.

Questions you may be asked

Did you train an AI? No. We use Google's pretrained hand and arm models and build everything after them ourselves.

How do you get 3D from one camera? The hand gives 3D points from a pretrained model. For object shape we use three views (front, side, top) and a visual hull. Depth in millimetres needs two cameras.

How is a pinch detected? Thumb-to-index distance divided by palm size is below 0.30; it stays pinched until it passes 0.45.

Why divide by palm size? So the gesture works at any distance and for any hand size.

Why is the cursor smoothed? Hands tremble and cameras are noisy. The One-Euro filter smooths slow moves and stays quick for fast ones.

What is a visual hull? The intersection of the 3D tubes made by pulling each silhouette through space. It always contains the object.

Why is a hull too big? It cannot remove dents or the space between views. A ball from 2 views is 27% too big, from 3 views 12%.

How do you find the object in a photo? Estimate the background, mark pixels that differ (Otsu threshold), keep the biggest blob, fill holes, trace the edge.

How do you measure quality? IoU for outlines (about 0.98), volume error for the hull, wobble and false pinches for tracking.

What is hysteresis? Two thresholds, one to turn on and a higher one to turn off, so a gesture does not flicker.

How is it tested? 265 JavaScript and 23 Python tests, using simulated hands and pictures, so no camera is needed.

What would you do next? Colour and depth for faces, a Sobel edge mode, stereo with two webcams, and a learned gesture classifier.

What are the numbers to remember? 21 landmarks; pinch 0.30 / 0.45; focal 1.35 × height; 40.6° field of view; 288 tests; hull error +27% / +12%; IoU 0.98; wobble 4.9 → 1.2; false pinches 75 → 0.